

CHALKBOX

Technical Documentation Report

AI Lesson Planning and Classroom Delivery Platform for Class 8-10 Science

Your classroom. Your plan. Powered by HarshLabs AI.

Project: Omnicon National Hackathon 2026 - Final Round

Team: HarshLabs

Prepared by: Harshawardhan Chitnis

Production: <https://chalkbox-harshlabs.pages.dev>

Repository: https://github.com/harshawardhanchitnis/Omnicon_Hackathon_Harshawardhan_Chitnis

Application release baseline: 063c0b4

Documentation baseline: main after README documentation commit 2a72168c

Date: 2 September 2026

Document Control

This report is the consolidated technical documentation for the ChalkBox production release. It describes the application architecture, data model, AI pipelines, security model, deployment, testing strategy and engineering roadmap as implemented in the public GitHub repository.

Field	Value
Document title	ChalkBox Technical Documentation Report
Project	ChalkBox - AI Lesson Planning for Real Classrooms
Competition	Omnikon National Hackathon 2026 - Final Round
Team	HarshLabs
Production URL	https://chalkbox-harshlabs.pages.dev
Repository	harshawardhanchitnis/Omnikon_Hackathon_Harshawardhan_Chitnis
Frontend release baseline	Commit 063c0b4 - final Round 3 certification blockers
Documentation baseline	README documentation commit 2a72168c
Primary scope	Class 8-10 Science lesson planning and classroom delivery

Documentation intent	This document is a technical implementation report, not a formal curriculum certification, penetration-test report, or guarantee of third-party AI availability.
-----------------------------	--

Executive Summary

ChalkBox is a teacher-first Generative AI application that converts either a textbook source or an unseen Science teaching request into a structured, classroom-ready teaching plan. The platform is deliberately designed around the workflow of a teacher rather than the conversational format of a generic chatbot.

The application combines a React/TypeScript frontend, Supabase authentication and data services, PostgreSQL with pgvector, private PDF storage, Deno Edge Functions, Google Gemini generation and embeddings, deterministic guardrails, independent science/safety auditing, multilingual rendering, and Cloudflare Pages production hosting.

The product provides a verified public demo and an authenticated teacher workspace. In the authenticated path, teachers can generate live Topic Mode lessons, upload private textbooks, retrieve source-grounded evidence using vector similarity, persist lesson plans to a cloud library, and reopen classroom experiences such as Full Lesson, Start Class, Quick Teach and Present Mode.

Core engineering principle	AI output is not treated as the final product. ChalkBox adds structure, validation, source provenance, safety controls, resource adaptation and teacher-facing delivery experiences around model output.
-----------------------------------	--

Key technical characteristics

- Frontend: React 19, TypeScript 6, Vite 8, React Router, Tailwind CSS 4, Base UI/shadcn components.
- Backend: Supabase Auth, PostgreSQL, Storage and Deno Edge Functions.
- RAG: 768-dimensional Gemini embeddings stored in pgvector with HNSW cosine indexes.
- AI control plane: model fallback routing, deterministic preflight rules, structural validation, independent audit and fail-closed behavior.
- Security: Row Level Security, owner-scoped teacher data, private storage, server-side AI credentials and no student account requirement.
- Deployment: Cloudflare Pages frontend with Supabase serverless/backend services.

Table of Contents

Document Control

2

Executive Summary

3

Key technical characteristics

3

1. Introduction

7

1.1 Project Context

7

1.2 Problem Statement

7

1.3 Objectives

7

1.4 Scope and Non-Goals

7

2. Product and System Requirements

8

2.1 Primary User

8

2.2 Functional Requirements

8

2.3 Non-Functional Requirements

8

3. System Overview and High-Level Architecture

9

3.1 Architectural Layers

9

3.2 Separation of Public Demo and Authenticated Product

9

4. Frontend Architecture

10

4.1 Technology

10

4.2 Route Map

10

4.3 Lesson Experience Model

10

4.4 Canonical Teaching Sequence

10

4.5 Responsive and Classroom-Oriented UI

11

5. Backend and Serverless Architecture

12

5.1 Supabase Responsibilities

12

5.2 Edge Function Inventory

12

5.3 Authentication and Session Handling

12

5.4 API Error Handling

12

6. Data Architecture

13

6.1 Authenticated Product Tables

13

6.2 Prepared Demo Corpus

13

6.3 Vector Search

13

6.4 Storage Model

13

7. AI Generation and Retrieval-Augmented Generation

14

7.1 Live Topic Mode Pipeline

14

Complete Lesson Mode

14

Focused Help Mode	14
Model routing	14
7.2 Private Textbook RAG Pipeline	14
Provenance	14
Completeness validation	14
7.3 Hindi Rendering	15
8. Safety, Science Accuracy and Trust Controls	16
8.1 Deterministic Preflight	16
8.2 Science-Specific Precision Rules	16
8.3 Independent Audit	16
8.4 Safety Scoping	16
8.5 Trust Boundaries	16
9. End-to-End Workflows	17
9.1 Verified Demo Workflow	17
9.2 Authenticated Topic Workflow	17
9.3 Private Textbook Workflow	17
9.4 Lesson Delivery Workflow	17
10. Application Interfaces and Contracts	18
10.1 Topic Request Contract	18
10.2 Browser Environment Variables	18
10.3 Server-Side Environment	18
10.4 Build Commands	18
11. Deployment and DevOps	19
11.1 Production Topology	19
11.2 Cloudflare Pages Build	19
11.3 Supabase Function Deployment	19
11.4 Release Gates	19
11.5 Release Identity	19
12. Testing and Release Validation	20
12.1 Validation Strategy	20
12.2 Functional Test Areas	20
12.3 Post-Fix Targeted QA Outcome	20
12.4 Remaining Hardening Areas	20
13. Performance, Reliability and Error Handling	21
13.1 External Provider Resilience	21
13.2 Fail-Closed Behavior	21
13.3 Deterministic Demo Reliability	21

13.4 Client Cancellation and Long Operations 21

13.5 Document Ingestion 21

14. Security and Privacy

22

14.1 Row Level Security 22

14.2 Private Storage 22

14.3 Server-Side Secrets 22

14.4 Protected Retrieval 22

14.5 Student Privacy 22

14.6 Threat Model Summary 22

15. Known Limitations and Technical Debt

23

16. Future Roadmap

24

17. Repository Structure and Developer Guide

25

17.1 Local Development 25

17.2 Production Build 25

17.3 Environment Safety 25

18. Conclusion

26

Appendix A. Deployment and Setup Checklist

27

A.1 Frontend 27

A.2 Backend 27

A.3 Release Smoke Test 27

Appendix B. Glossary

28

Appendix C. Reference Links and Repository Sources

29

1. Introduction

1.1 Project Context

ChalkBox was built for the Omnicon National Hackathon 2026 Final Round as a practical classroom planning system. The project addresses a common gap between having curriculum material and being ready to teach it: teachers still need to decide what to explain first, what to write on the board, which example to use, which activity is feasible, how to check understanding and how to adapt to the resources actually available.

The platform therefore treats Generative AI as a planning and transformation engine inside a constrained teacher workflow. The output is shaped into explicit teaching stages and multiple delivery surfaces rather than returned as a single long answer.

1.2 Problem Statement

Generic AI systems can generate explanations, but classroom use introduces additional requirements: age appropriateness, safe activity design, source faithfulness, limited resources, timing, projector readability, multilingual delivery, persistent lesson management and teacher control. ChalkBox is designed to combine these requirements in one workflow.

1.3 Objectives

- 1 Reduce teacher preparation effort by turning a topic or textbook source into a structured lesson plan.
- 2 Keep lesson generation constrained to Class 8-10 Science and to realistic classroom delivery.
- 3 Support both complete lesson planning and focused help for one difficult concept.
- 4 Ground private-textbook lessons in retrieved source content with visible page provenance.
- 5 Adapt activities and teaching approaches to low-resource or well-equipped classroom contexts.
- 6 Provide safe, scientifically accurate outputs through layered deterministic and AI-based checks.
- 7 Give teachers reusable classroom views: Full Lesson, Start Class, Quick Teach and Present Mode.
- 8 Persist authenticated teacher plans and private textbooks securely in a cloud workspace.

1.4 Scope and Non-Goals

In scope	Out of scope / non-goal
Class 8-10 Science lesson planning	General-purpose chatbot functionality
Teacher-facing planning and delivery	Student social network or student account system
Public verified demo	Automatic grading of individual students
Live Topic generation	Unrestricted subject generation
Private textbook RAG	Public redistribution of teacher-uploaded textbook content
Hindi lesson rendering	Full multilingual coverage for all Indian languages

2. Product and System Requirements

2.1 Primary User

The primary user is a school Science teacher who needs planning support before class or compact guidance during class. Judges and visitors form a secondary public-demo audience. The authenticated product is teacher-centric and does not require student accounts.

2.2 Functional Requirements

- Allow a public user to explore prepared demo lessons without registration.
- Allow an authenticated teacher to register, sign in, refresh a session, sign out and recover a password.
- Generate live Topic Mode lessons from unseen Science requests.
- Support Complete Lesson and Focused Help request modes.
- Allow resource-aware generation and configurable lesson duration, language and classroom context.
- Upload teacher-owned PDF textbooks to private storage and index them for retrieval.
- Generate textbook-grounded lessons from only the selected private document.
- Persist plans, notes, customizations and source documents in the teacher workspace.
- Render lesson content into Full Lesson, Start Class, Quick Teach and Present Mode experiences.
- Provide Hindi rendering through a server-side translation workflow.
- Reject out-of-scope or obviously dangerous requests before unsafe content is generated.

2.3 Non-Functional Requirements

Requirement	Implementation approach
Security	Supabase Auth, RLS, private storage, server-side secrets, owner-scoped data
Reliability	Prepared demo fallback, generation retries, model fallback routing, fail-closed audits
Usability	Teacher-first flow, explicit stages, responsive layouts, compact classroom views
Traceability	Source-page provenance for textbook-grounded generation
Maintainability	React component separation, Edge Function boundaries, migration-driven schema
Deployability	Vite production build, Cloudflare Pages, Supabase serverless functions
Privacy	No student account requirement; teacher-owned textbook data is private

3. System Overview and High-Level Architecture

ChalkBox uses a split architecture: a static React SPA is served globally through Cloudflare Pages, while authenticated data, private files, vector search and AI orchestration run through Supabase. Google Gemini is called only from server-side functions for generation, auditing, translation support and embeddings.

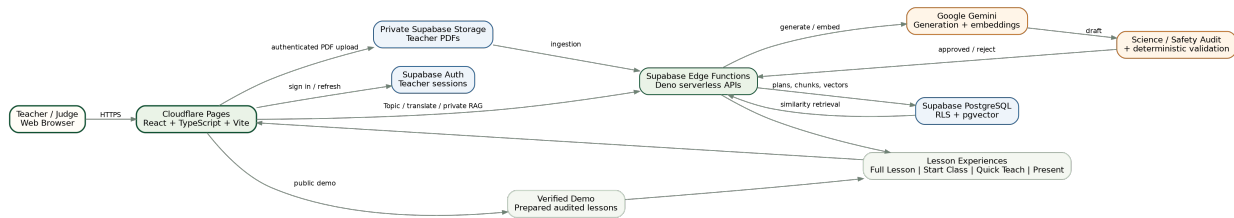


Figure 1. High-level ChalkBox production architecture.

3.1 Architectural Layers

Layer	Responsibility
Presentation	React pages, lesson views, navigation, responsive UI, classroom presentation controls
Client domain logic	Topic request handling, lesson formatting, language rendering, local draft/history state, cancellation
Identity and persistence	Supabase Auth, teacher sessions, cloud lesson library, notes and customizations
Private content	Supabase Storage for PDFs; page/chunk tables for indexed source content
AI orchestration	Supabase Edge Functions for generation, auditing, ingestion, translation and private RAG
Retrieval	PostgreSQL pgvector similarity search scoped to a specific document and owner
External AI	Gemini generation models and 768-dimensional embedding model
Hosting	Cloudflare Pages for the frontend and Supabase for serverless/backend services

3.2 Separation of Public Demo and Authenticated Product

The verified demo is intentionally deterministic and independent of live AI availability. Authenticated Product Mode uses live generation, private storage and cloud persistence. This separation improves judge/demo reliability while preventing prepared demo data from being substituted for a failed live Product request.

4. Frontend Architecture

4.1 Technology

The frontend is a React 19 single-page application written in TypeScript 6 and bundled with Vite 8. Routing uses React Router. UI composition uses Tailwind CSS 4, Base UI/shadcn-oriented components and Lucide icons. Inter is used as the primary application font.

4.2 Route Map

Route	Access	Purpose
/	Public	Landing page and entry point
/login	Public	Teacher sign-in
/register	Public	Teacher registration
/forgot-password	Public	Password recovery request
/reset-password	Public	Recovery-session password update
/dashboard	Public/demo	Demo dashboard
/textbook	Public/demo	Verified textbook demo selection
/topic	Public + product parameter	Topic Mode request form
/topic/lesson	Public/demo	Topic lesson workspace
/lesson/:lessonKey	Public/demo	Prepared lesson workspace
/app	Authenticated	Teacher product home
/app/library	Authenticated	Cloud lesson library
/app/textbooks	Authenticated	Private textbook management
/app/textbooks/:documentId/generateLesson	Authenticated	Private textbook lesson generation
/app/lesson/:planId	Authenticated	Saved product lesson

4.3 Lesson Experience Model

A generated or prepared lesson is transformed into multiple teacher-facing views. This is a core design decision: the same pedagogical content is adapted to different classroom moments.

Experience	Use case
Full Lesson	Complete planning/reference view with board plan, teaching guidance, activity, practice and materials
Start Class	Step-by-step classroom delivery with canonical teaching stages and timing
Quick Teach	Condensed essential teaching path for fast use while retaining relevant safety guidance
Present Mode	Fullscreen projector view with stage numbering, continuation pages, timer and language support

4.4 Canonical Teaching Sequence

Hook -> Define -> Explain -> Visualize -> Example -> Activity -> Practice -> Check Understanding

Supporting content includes Board Plan, How to Teach guidance, Materials, teacher cues, expected responses, formulas, safety notes and source pages where applicable.

4.5 Responsive and Classroom-Oriented UI

The frontend is designed for desktop, tablet and mobile use, with a separate compact authenticated navigation layout at narrower widths. Present Mode is optimized for projector use and uses continuation pages for dense lesson content. Exact dense-visual pagination remains an active hardening area documented later in this report.

5. Backend and Serverless Architecture

5.1 Supabase Responsibilities

- Authentication: teacher identity, access tokens, refresh tokens and recovery sessions.
- PostgreSQL: teacher plans, notes, customizations, source documents, pages, chunks and usage events.
- Storage: private PDF bucket for teacher-owned textbooks.
- Edge Functions: AI orchestration, ingestion, audit and translation.
- pgvector: similarity search over textbook chunks.

5.2 Edge Function Inventory

Function	Responsibility
generate-topic-lesson	Generate Complete or Focused Topic lesson drafts with deterministic request validation
audit-topic-lesson	Independent review of science accuracy, age appropriateness, request fit, feasibility and safety
ingest-private-textbook	Process teacher PDF content and prepare pages/chunks/embeddings for private retrieval
generate-private-textbook-lesson	Retrieve document-scoped evidence and generate a source-grounded private lesson
translate-lesson-text	Server-side translation support for Hindi lesson rendering

5.3 Authentication and Session Handling

The client reads public Supabase configuration from Vite environment variables, authenticates against Supabase Auth, stores the teacher session locally, refreshes near-expiry sessions and clears state on failed refresh or sign-out. Password recovery uses a configured application origin to construct the reset redirect.

Protected product routes are wrapped by an authenticated-route component. Backend product functions validate the bearer token against Supabase before executing owner-specific operations.

5.4 API Error Handling

The client maps common backend conditions to teacher-readable messages. Network errors and hosted worker termination conditions can be retried. AI quota or service failures are surfaced without substituting prepared demo content into Product Mode.

6. Data Architecture



Figure 2. Simplified authenticated product data model and prepared demo corpus.

6.1 Authenticated Product Tables

Table	Purpose / important fields
teacher_plans	Owner-scoped saved Topic/private-textbook plans; source mode/id, title, class, duration, language, resource_level, payload
teacher_notes	Owner-scoped notes keyed to a lesson source identity
lesson_customizations	Owner-scoped JSON patch data for lesson customization
source_documents	Uploaded PDF metadata, storage path, status, page counts and error metadata
source_pages	Extracted page text, page number, readability and extraction method
source_chunks	Document chunks with page ranges and vector(768) embeddings
ai_usage_events	Server-managed usage events for daily limit enforcement

6.2 Prepared Demo Corpus

The repository also contains a separate prepared textbook lesson catalog and backend RAG foundation tables for the six supported demo lessons. The demo corpus is kept separate from teacher-owned private documents.

6.3 Vector Search

Both prepared and private textbook retrieval use 768-dimensional vectors with cosine-distance operators. PostgreSQL HNSW indexes are used to accelerate similarity search as the corpus grows.

6.4 Storage Model

The product migration creates a private storage bucket named teacher-textbooks with PDF-only access and a 25 MB file-size limit. Storage policies require the first folder segment to match the authenticated teacher user ID.

7. AI Generation and Retrieval-Augmented Generation

7.1 Live Topic Mode Pipeline

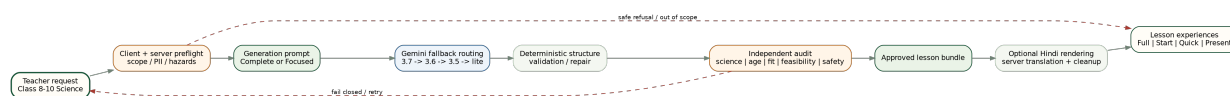


Figure 3. Live Topic Mode generation, validation and audit pipeline.

Topic Mode accepts Class 8, 9 or 10 Science requests with Complete or Focused modes, duration, resource level, language and optional classroom context. The backend schema retains a legacy `standard` resource value for compatibility, while the product UI is designed around Low-resource and Well-equipped choices.

Complete Lesson Mode

Complete Mode builds a full classroom-ready sequence and supporting planning material. Complete requests require sufficient available lesson time and are validated before use.

Focused Help Mode

Focused Mode intentionally avoids expanding into a full chapter lesson. It returns a compact intervention containing a teaching goal, board plan, simple explanation, visual/analogy, example, micro-activity, misconception correction, quick check and reteaching moves.

Model routing

The generation path uses fallback routing across configured Gemini Flash models. The current code attempts a preference order that includes Gemini 3.7 Flash, 3.6 Flash, 3.5 Flash and a 3.5 Flash Lite fallback. Provider errors such as quota and transient service failures are handled without replacing live content with prepared demo lessons.

7.2 Private Textbook RAG Pipeline



Figure 4. Private teacher-textbook retrieval and grounded generation pipeline.

The private RAG path is document-scoped and owner-scoped. A teacher uploads a PDF, readable pages are extracted, the content is chunked, and Gemini embeddings are generated with an output dimensionality of 768. Chunks are stored in pgvector and retrieved using cosine similarity only for the selected document.

Provenance

Retrieved page ranges are retained through generation so lesson sections can expose supporting physical source-page references. This provides a practical verification path for the teacher and reduces the risk of invisible source drift.

Completeness validation

Before a private Complete lesson is accepted, required lesson sections are checked deterministically. An incomplete draft can be regenerated once; if it remains incomplete, the request fails closed rather than saving a misleading truncated lesson.

7.3 Hindi Rendering

Canonical generation is performed in English. When Hindi is selected, lesson text is translated through a server-side translation function and then passed through targeted cleanup/normalization rules for known classroom terminology and malformed fragments. This keeps generation and language transformation as separate concerns.

8. Safety, Science Accuracy and Trust Controls

ChalkBox uses layered controls because classroom safety and scientific precision cannot be delegated to a single prompt. The architecture combines deterministic request checks, generation constraints, post-generation normalization, independent auditing and view-level safety rendering.

8.1 Deterministic Preflight

- Rejects non-Science requests such as Mathematics, History/Social Science and language/literature requests.
- Rejects obvious hazardous home experiments involving strong acids, open flames, unsafe heating or improvised glassware.
- The frontend also applies fast deterministic guards to known dangerous sunlight-focusing and hard-projectile-release patterns.
- Topic input logic checks for obvious PII patterns such as email addresses and phone numbers.

8.2 Science-Specific Precision Rules

The Topic generator includes targeted correctness rules for recurring school-science misconceptions, including magnesium oxide/magnesium hydroxide chemistry, metal-oxide generalizations, buoyancy equilibrium, Rutherford scattering language and misleading density analogies. Known problematic wording can also be normalized after generation.

8.3 Independent Audit

Generated Topic drafts are separately audited for science accuracy, age appropriateness, request fit, classroom feasibility and safety. Private textbook lessons add source faithfulness to the audit criteria. The application fails closed when a draft cannot be approved.

8.4 Safety Scoping

Safety notes are attached to the relevant Activity or Visualize content rather than blindly copying hazard text across an entire lesson. Quick Teach contains fallback practical guidance for known material risks such as sharp foil edges and water spills.

8.5 Trust Boundaries

Trust boundary	Control
Model output	Schema/structure validation, science normalization, independent audit
Teacher PDF	Private storage, owner-scoped rows, selected-document retrieval
Browser	Only public Supabase keys; no Gemini/service-role secrets
Saved lesson	Owner-scoped RLS and cloud library
Public demo	Prepared deterministic lesson data, separate from product data

9. End-to-End Workflows

9.1 Verified Demo Workflow

- 1 Visitor opens the public landing page.
- 2 Visitor enters the verified demo without registration.
- 3 A prepared Class 8, 9 or 10 Science lesson is selected.
- 4 The same prepared lesson can be opened in Full Lesson, Start Class, Quick Teach and Present Mode.
- 5 The workflow remains usable even if external AI services are unavailable.

9.2 Authenticated Topic Workflow

- 1 Teacher signs in and opens Product Topic Mode.
- 2 Teacher selects class, duration, resource level, language, mode and provides a teaching request.
- 3 Deterministic scope/safety checks run before live generation.
- 4 The generation function creates a Complete or Focused draft using model fallback routing.
- 5 Structural validation and independent audit approve or reject the draft.
- 6 The approved plan is persisted to the teacher cloud library.
- 7 The teacher opens the lesson through Full Lesson, Start Class, Quick Teach or Present Mode.

9.3 Private Textbook Workflow

- 1 Teacher uploads a PDF to private storage.
- 2 The ingestion pipeline extracts readable pages, chunks content and creates vector embeddings.
- 3 The teacher asks for a lesson from that specific document.
- 4 Similarity retrieval selects the most relevant document chunks.
- 5 The private generation function produces a lesson constrained by the retrieved source.
- 6 Completeness, source faithfulness, science and safety are checked.
- 7 The approved lesson is saved with source-page provenance and can be reopened from the Library.

9.4 Lesson Delivery Workflow

A lesson bundle is not locked to a single page. Teachers can move between planning, live delivery, quick-reference and projector-oriented experiences. Notes and cloud persistence allow the lesson to be reopened after refresh or on a later visit.

10. Application Interfaces and Contracts

10.1 Topic Request Contract

```
{
  "classLevel": 9,
  "subject": "Science",
  "requestMode": "complete",
  "teacherRequest": "Explain Rutherford scattering and why most alpha particles pass straight through.",
  "durationMinutes": 40,
  "resourceLevel": "low",
  "language": "english",
  "classroomContext": "No projector; chalkboard available"
}
```

10.2 Browser Environment Variables

Variable	Purpose
VITE_SUPABASE_URL	Public Supabase project URL
VITE_SUPABASE_PUBLISHABLE_KEY or VITE_SUPABASE_ANON_KEY	Supabase Publishable key used for Auth/REST requests
VITE_APP_URL	Application origin used for recovery redirects
VITE_TOPIC_GENERATION_ENDPOINT (optional)	Override for Topic generation Edge Function
VITE_TOPIC_AUDIT_ENDPOINT (optional)	Override for Topic audit Edge Function

10.3 Server-Side Environment

Edge Functions require Supabase server configuration and Gemini credentials. Server secrets include the Gemini API key and Supabase service-role or secret key; these must never be placed in browser `.env`` files.

10.4 Build Commands

```
pnpm install
pnpm dev
pnpm exec tsc -b
pnpm build
```

11. Deployment and DevOps

11.1 Production Topology

Component	Production service
Frontend SPA	Cloudflare Pages
Production domain	https://chalkbox-harshlabs.pages.dev
Source control	GitHub - main branch
Authentication / DB / Storage	Supabase
Serverless backend	Supabase Edge Functions
AI provider	Google Gemini

11.2 Cloudflare Pages Build

The production Pages project builds from the GitHub `main` branch. The configured build command is `pnpm build` and the output directory is `dist`. The production environment uses Node 22.17.0 and pnpm 10.34.5.

11.3 Supabase Function Deployment

```
pnpm dlx supabase@latest functions deploy <function-name> --project-ref <project-ref>
```

Frontend pushes and Supabase function deployments are independent. A frontend-only change requires a Cloudflare rebuild; an Edge Function change requires a Supabase function deployment.

11.4 Release Gates

- `git diff --check` for whitespace/patch integrity.
- `pnpm exec tsc -b` for TypeScript compilation.
- `pnpm build` for the production bundle.
- Repository secret scanning script before commit/push.
- Hosted smoke tests for public, authenticated, private-RAG, Hindi, responsive and safety flows.

11.5 Release Identity

The final hackathon application release was pushed at commit 063c0b4 with the message fix: close final Round 3 certification blockers. A subsequent documentation-only commit updated the README without changing application logic.

12. Testing and Release Validation

12.1 Validation Strategy

Testing combined local build gates with hosted adversarial browser testing. The release test plan covers authentication, cloud persistence, private textbooks, RLS expectations, deployment, responsive layouts, print behavior, language rendering and logout.

12.2 Functional Test Areas

Area	Examples
Public demo	Six prepared lessons, Full Lesson, Start Class, Quick Teach, Present
Topic generation	Complete, Focused, safe request, out-of-scope request, cancellation
Private RAG	Upload, ingestion, generation, provenance, save/reopen, delete
Authentication	Register, sign in, refresh, sign out, recovery
Responsive UI	Desktop, tablet, mobile, authenticated header, projector view
Safety	Acid/flame, direct sunlight, projectile release, sharp objects, hydrogen ignition risk
Science accuracy	Buoyancy, magnesium chemistry, Rutherford scattering, indicator behavior
Localization	Hindi rendering, terminology cleanup, voice controls

12.3 Post-Fix Targeted QA Outcome

A targeted hosted retest on 1 September 2026 verified the new production flow after the final blocker patch. The following items were confirmed closed in that retest:

- Private 40-minute textbook lesson truncation: complete lesson structure persisted and reopened correctly.
- Private Activity page provenance omission: supporting physical pages were present in the retested source-grounded Activity.
- Authenticated 768 px page-wide overflow.
- Hindi malformed fragments such as the previously observed path-term corruption.
- Irrelevant chemistry safety leakage into a harmless biology activity.
- Foil/water Quick Teach practical safety guidance.
- Deterministic acid/flame, direct-sunlight and hard-projectile refusals, all rejecting near-immediately.

12.4 Remaining Hardening Areas

The same targeted retest identified two UI/safety consistency areas that remain useful engineering work after submission: dense Product Present visual slides can still exceed the intended projector frame in some cases, and a prepared potato-cutting safety transformation is not yet propagated identically to every live-teaching surface. These are documented as technical debt rather than hidden.

13. Performance, Reliability and Error Handling

13.1 External Provider Resilience

The AI layer can encounter quota exhaustion, transient HTTP failures and hosted worker termination. ChalkBox uses model fallback routing and selected retries for retryable conditions. User-facing errors distinguish provider unavailability from normal application behavior.

13.2 Fail-Closed Behavior

For unsafe, structurally incomplete or unapproved content, the preferred behavior is to reject or request retry rather than save a misleading lesson. This is particularly important for private textbook lessons, where a truncated result could appear authoritative because it is source-grounded.

13.3 Deterministic Demo Reliability

The prepared demo ensures that the full classroom workflow remains demonstrable even when external AI services are unavailable. This reduces the coupling between product demonstration and provider quotas.

13.4 Client Cancellation and Long Operations

Live generation uses abortable network requests so a teacher can cancel a legitimate long-running request. The design intentionally avoids artificially short timeouts that would make valid generation unreliable.

13.5 Document Ingestion

Private textbook ingestion works best for normal-size selectable-text PDFs. Very large books are a known scalability area; resumable batching and recovery checkpoints are planned for future work.

14. Security and Privacy

14.1 Row Level Security

Teacher plans, notes, customizations, source documents, pages and chunks are protected by RLS policies keyed to ``auth.uid()``. The authenticated user can only operate on their own rows through normal browser access.

14.2 Private Storage

The ``teacher-textbooks`` bucket is private. Storage policies restrict object access to a path namespace owned by the authenticated teacher. Teacher PDFs are not public assets.

14.3 Server-Side Secrets

Gemini credentials and Supabase service-role/secret credentials are read only inside Edge Function environments. The browser receives only the public Supabase URL and publishable/anonymous key.

14.4 Protected Retrieval

Private similarity retrieval is executed with an authenticated context and document ID. The retrieval function filters by both ``auth.uid()`` and selected document, preventing cross-teacher or cross-document retrieval under normal application access.

14.5 Student Privacy

The current product is teacher-first and does not require student accounts or student PII. Topic input logic additionally guards against obvious email and phone patterns.

14.6 Threat Model Summary

Threat	Mitigation
Cross-teacher data access	RLS + owner_id checks + private storage paths
API secret leakage	Server-side Edge Function secrets only
Prompt asks for unsafe procedure	Deterministic refusal + safety prompt rules + audit
Hallucinated textbook provenance	Document-scoped RAG + source-page sanitation + audit
Incomplete saved lesson	Completeness validation and fail-closed retry
External AI outage/quota	Prepared demo, fallback routing, explicit error handling

15. Known Limitations and Technical Debt

A complete technical report should distinguish current capability from planned hardening. The following limitations are known and are consistent with the project roadmap.

Area	Current limitation / technical debt
Very large PDF ingestion	Needs resumable batching, checkpointing and more graceful recovery under embedding/provider
Dense projector visuals	Highly visual generated slides can still require better adaptive pagination at 1366x768
Safety wording consistency	Prepared safety transformations should be centralized so every teaching surface renders the sa
External AI dependency	Live generation and embeddings remain subject to third-party quota and latency
Subject scope	Production scope is intentionally limited to Class 8-10 Science
Language scope	Hindi is supported; broader multilingual support is future work
Curriculum adapters	The platform is board-neutral in product wording, but broader curriculum adapter tooling can be

16. Future Roadmap

- Resumable textbook ingestion: chunk-level checkpoints, batching and restart-safe indexing.
- Adaptive presentation layout: slide-level measurement and automatic visual/text repartitioning for projector resolutions.
- Centralized safety rendering: one transformation layer shared by Full Lesson, Start Class, Quick Teach and Present Mode.
- Additional subjects: controlled expansion beyond Science after subject-specific audit rules are defined.
- Curriculum adapters: configurable board/curriculum metadata without hard-coding a single provider into the product experience.
- Multilingual expansion: additional Indian languages with terminology QA.
- Teacher analytics: lesson usage, curriculum coverage and planning patterns without student surveillance.
- Collaborative workspaces: school/department sharing with explicit permissions and provenance.
- Richer customization: editable lesson variants while preserving source references and auditability.

17. Repository Structure and Developer Guide

```
Omnikon_Hackathon_Harshawardhan_Chitnis/  
|-- public/                # branding and illustration assets  
|-- src/  
|   |-- components/        # lesson, layout and product UI  
|   |-- data/              # prepared demo lesson/catalog data  
|   |-- hooks/             # product/session hooks  
|   |-- lib/               # auth, lesson, API, language and presentation logic  
|   |-- pages/             # public, demo and authenticated routes  
|-- supabase/  
|   |-- functions/         # AI/product Edge Functions  
|   |-- migrations/        # PostgreSQL, pgvector, RLS and product schema  
|-- scripts/               # verification/build utilities  
|-- PRODUCT_RELEASE_TEST_PLAN.md  
|-- package.json  
|-- README.md
```

17.1 Local Development

```
git clone https://github.com/harshawardhanchitnis/Omnikon_Hackathon_Harshawardhan_Chitnis.git  
cd Omnikon_Hackathon_Harshawardhan_Chitnis  
pnpm install  
cp .env.example .env.local  
pnpm dev
```

17.2 Production Build

```
pnpm exec tsc -b  
pnpm build
```

The Vite bundle is emitted to `dist/` and is suitable for Cloudflare Pages deployment.

17.3 Environment Safety

Do not expose server secrets

Never place GEMINI_API_KEY, Supabase service-role keys or other backend secrets in `.env.local` browser configuration. Only public Supabase configuration belongs in Vite environment variables.

18. Conclusion

ChalkBox demonstrates a production-oriented way to apply Generative AI to classroom planning: the model is only one part of the system. The final product combines structured teaching design, vector retrieval, source provenance, safety controls, science-specific normalization, independent auditing, language transformation, authentication, cloud persistence and classroom presentation.

The architecture is deliberately teacher-first. It does not ask the teacher to interpret a generic AI answer; it converts a request into a sequence that can be planned, taught, shortened, presented and reopened. The deterministic verified demo, private textbook RAG path and fail-closed validation strategy also make the system more resilient and trustworthy than a single-call prototype.

The remaining roadmap is clear: strengthen large-document ingestion, centralize safety transformations across all views, improve adaptive projector pagination, expand language/curriculum adapters and broaden the subject scope only when equivalent accuracy and safety controls are available.

ChalkBox - Plan better lessons. Teach with confidence.

Appendix A. Deployment and Setup Checklist

A.1 Frontend

- Node.js 22.x and pnpm 10.x installed.
- Public Supabase URL and publishable key configured.
- VITE_APP_URL set to the deployment origin.
- `pnpm build` completes successfully.
- Cloudflare Pages points to the `main` branch and `dist` output.

A.2 Backend

- Supabase migrations applied.
- Required Edge Functions deployed.
- Gemini API key configured only in server environment.
- Supabase service secret configured only server-side.
- Private storage bucket and RLS policies verified.

A.3 Release Smoke Test

- Public landing and verified demo open in incognito.
- Sign-in and authenticated `/app` routes work.
- A normal Topic request generates and saves.
- A private textbook can be uploaded, indexed, used and reopened.
- Hindi rendering completes.
- Unsafe prompts refuse without dangerous instructions.
- Responsive layouts and projector mode are spot-checked.

Appendix B. Glossary

Term	Meaning in ChalkBox
RAG	Retrieval-Augmented Generation: retrieve source evidence before generating a lesson
Embedding	Numerical vector representation used for semantic similarity search
pgvector	PostgreSQL extension used to store/query embeddings
HNSW	Approximate-nearest-neighbor index used to accelerate vector search
RLS	Row Level Security: database policy layer restricting rows by authenticated owner
Edge Function	Serverless Deno function hosted by Supabase
Complete Mode	Full Topic lesson generation path
Focused Mode	Compact teaching intervention for one specific teaching problem
Provenance	Source-page references supporting textbook-grounded lesson content
Fail closed	Reject/ask for retry instead of accepting an unsafe, unapproved or incomplete result

Appendix C. Reference Links and Repository Sources

Live application: <https://chalkbox-harshlabs.pages.dev>

GitHub repository:
https://github.com/harshawardhanchitnis/Omnikon_Hackathon_Harshawardhan_Chitnis

Primary repository artifacts used to prepare this documentation:

- ``README.md`` - product overview, architecture, setup, testing and roadmap.
- ``package.json`` - framework and library versions plus build scripts.
- ``src/App.tsx`` - public/demo/authenticated route map.
- ``src/pages/LandingPage.tsx`` and product pages - user-facing product definition.
- ``src/lib/productAuth.ts``, ``src/lib/productApi.ts``, ``src/lib/topicMode.ts`` - auth and client orchestration.
- ``supabase/functions/*`` - generation, auditing, ingestion, private RAG and translation services.
- ``supabase/functions/_shared/product.ts`` - Supabase/Gemini helpers, rate limiting and embeddings.
- ``supabase/migrations/*`` - PostgreSQL schema, pgvector indexes, RLS policies and private storage configuration.
- ``PRODUCT_RELEASE_TEST_PLAN.md`` - release validation checklist.
- ``ChalkBox Post-Fix Targeted Retest`` (1 Sep 2026) - hosted post-fix QA evidence used for the test-status summary.

Version note

Application behavior described here corresponds to the submitted production release baseline. The documentation-only README commit does not change application logic.

END OF TECHNICAL DOCUMENTATION